# PROJECT REPORT

## SCRIPT-CONTROLLED ACL – RESTRICT RECORD ACCESS BASED ON FIELD VALUE

**Submitted by**

Sivasakthi.K (Team Leader)

B. TECH INFORMATION TECHNOLOGY

Sri Ramanujar Engineering College

Academic Project Documentation

# ABSTRACT

This project presents a script-controlled Access Control List (ACL) mechanism for restricting access to individual records according to the value of a selected field. Traditional record-level access rules often depend on static role or group permissions. Such rules can become difficult to maintain when access must change dynamically based on record attributes. The proposed solution introduces a script-based authorization layer that evaluates a field value at the time a record is requested and permits or denies access according to configurable conditions.

The design separates authentication, record retrieval, policy evaluation, and authorization. A user may be authenticated successfully but still be denied access to a particular record when the record does not satisfy the applicable policy. This approach supports fine-grained authorization and can be adapted to employee, department, project, status, ownership, or classification fields.

The report follows the supplied project-report format and documents the problem, proposed solution, requirements, architecture, planning, testing, results, advantages, limitations, conclusion, future scope, and an illustrative source-code appendix.

# TABLE OF CONTENTS


| Section | Page |
|---|---|
| 1. INTRODUCTION | 3 |
| 1.1 Project Overview | 3 |
| 1.2 Purpose | 4 |
| 2. IDEATION PHASE | 5 |
| 2.1 Problem Statement | 5 |
| 2.2 Empathy Map Canvas | 5 |
| 2.3 Brainstorming | 6 |
| 3. REQUIREMENT ANALYSIS | 7 |
| 3.1 Customer Journey Map | 7 |
| 3.2 Solution Requirement | 8 |
| 3.3 Data Flow Diagram | 9 |
| 3.4 Technology Stack | 10 |
| 4. PROJECT DESIGN | 11 |
| 4.1 Problem Solution Fit | 11 |
| 4.2 Proposed Solution | 12 |
| 4.3 Solution Architecture | 13 |
| 5. PROJECT PLANNING & SCHEDULING | 14 |
| 5.1 Project Planning | 14 |
| 6. FUNCTIONAL AND PERFORMANCE TESTING | 16 |
| 6.1 Performance Testing | 16 |
| 7. RESULTS | 18 |
| 7.1 Output Screenshots | 18 |
| 8. ADVANTAGES & DISADVANTAGES | 19 |
| 9. CONCLUSION | 20 |
| 10. FUTURE SCOPE | 21 |
| 11. APPENDIX | 22 |

# 1. INTRODUCTION

## 1.1 Project Overview

The Script-Controlled ACL project is designed to implement fine-grained record access control based on a field value. An ACL is a rule that determines whether a user or group is allowed to perform an operation on a protected resource. In this project, the authorization decision is not based only on who the user is; it also considers information stored in the requested record.

For example, a policy can permit a user to read records whose department field matches the user's department, while denying records belonging to other departments. Similar rules can be created for ownership, project code, region, classification, status, or another business field. The script evaluates the condition before the requested record is exposed.

The proposed workflow is: authenticate the user → identify the requested record → read the relevant field → evaluate the ACL script → grant or deny access → log the decision. This creates a clear authorization boundary and makes the policy easier to adapt when business rules change.

### High-Level Workflow

| User Request | Authentication | Record + Field | ACL Script | Allow / Deny |
|---|---|---|---|---|

## 1.2 Purpose

The purpose of the project is to demonstrate a practical method for enforcing dynamic record-level security through a script-controlled ACL. The system aims to reduce unauthorized record exposure, provide consistent policy evaluation, and make access rules easier to understand and modify.

• Provide record-level authorization instead of relying only on broad application roles.

• Evaluate access using a selected field value and user attributes.

• Return a clear allow/deny decision before protected data is displayed.

• Support audit logging for authorization decisions.

• Provide a design that can be extended to additional fields and policy conditions.

# 2. IDEATION PHASE

## 2.1 Problem Statement

Many applications contain records that should not be visible to every authenticated user. A role such as 'employee' or 'manager' may be too broad because two users with the same role can legitimately need access to different records. Static permissions also become difficult to manage when the authorization condition depends on changing record attributes.

The problem addressed by this project is therefore: **How can an application restrict access to individual records dynamically by evaluating a field value against the authenticated user's permitted value?**

## 2.2 Empathy Map Canvas

| Says | Thinks | Does | Feels |
|---|---|---|---|
| “I should see only the records relevant to me.” | “Why can another department see this record?” | Logs in, searches for a record, requests access. | Concerned about privacy and unauthorized |
| “The rule should be easy to update.” | “Access should depend on the actual record.” | Reports incorrect access decisions. | Wants confidence in the security mechanis |

## 2.3 Brainstorming

• Use role-based access control as the basic authentication layer.

• Add field-based conditions for fine-grained record access.

• Evaluate the rule at request time rather than copying permissions into every record.

• Use reusable scripts for common authorization patterns.

• Log denied requests to support auditing and troubleshooting.

• Provide an administrator-friendly configuration model for changing policy conditions.

# 3. REQUIREMENT ANALYSIS

## 3.1 Customer Journey Map

| Stage | User Action | System Response | Possible Issue |
|---|---|---|---|
| Login | Submits credentials | Authenticates user | Invalid credentials |
| Select record | Requests a specific record | Identifies record and user context | Record not found |
| Policy evaluation | Waits for authorization | Reads field and evaluates ACL | Policy mismatch |
| Decision | Continues or receives denial | Allows or blocks request | Unauthorized access attempt |
| Audit | Reviews activity when needed | Stores decision details | Incomplete logs |

## 3.2 Solution Requirement

**Functional requirements**

• Authenticate the user before protected records are accessed.

• Identify the requested record and the field used for authorization.

• Retrieve the authenticated user's permitted value or attributes.

• Execute the ACL script against the record and user context.

• Allow access when the policy evaluates to true.

• Deny access when the policy evaluates to false.

• Record authorization outcomes for auditing.

• Return a safe error message without exposing protected record information.

**Non-functional requirements**

• Security: authorization must be enforced before record data is returned.

• Performance: policy evaluation should add minimal overhead.

• Maintainability: rules should be modular and readable.

• Scalability: the design should support many records and users.

• Reliability: failures in policy evaluation should not accidentally grant access.

• Auditability: decisions should be traceable.

## 3.3 Data Flow Diagram

## Level-1 Data Flow

| User | Application | ACL Engine | Record Store | Audit Log |
|---|---|---|---|---|

Data flow: the user sends a request to the application. The application identifies the user and requested record, obtains the relevant record field, and passes the context to the ACL engine. The ACL engine returns an allow/deny decision. Only an allowed request proceeds to the response stage, while the decision is written to the audit log.

## 3.4 Technology Stack

| Layer | Suggested Technology | Purpose |
|---|---|---|
| Client | HTML / CSS / JavaScript | Login/request interface and result display |
| Application | Python Flask or equivalent web framework | Request processing and API layer |
| Database | SQLite / MySQL / PostgreSQL | User, record and policy-related data |
| Authorization | Python ACL policy module | Dynamic field-value access decision |
| Logging | Application audit logger | Store allow/deny events |
| Testing | Python unittest / pytest | Functional and policy testing |
| Version control | Git / GitHub | Source-code management |

# 4. PROJECT DESIGN

## 4.1 Problem Solution Fit

| Problem | Proposed Fit |
|---|---|
| Broad roles expose too many records | Add record-field evaluation to the authorization decision. |
| Rules change with business conditions | Keep policy logic in a script/module that can be updated. |
| Users need only relevant records | Compare record field values with permitted user attributes. |
| Unauthorized requests need investigation | Store authorization decisions in an audit log. |
| Security failures must be safe | Default to deny when required policy data is missing or invalid. |

## 4.2 Proposed Solution

The proposed system introduces a policy function between record retrieval and record delivery. The policy receives a user context and record context. It checks whether the record's selected field satisfies the user's authorization condition. A true result produces an allow decision; a false result produces a deny decision.

Illustrative rule: if the record's *department* field is equal to the authenticated user's *department*, access is allowed. A stronger implementation can additionally require ownership, record status, role, or classification conditions.

## 4.3 Solution Architecture

## Logical Architecture

| Web / Client | Application API | Authentication | ACL Policy Engine | Database |
|---|---|---|---|---|

The client never directly decides authorization. The application is responsible for enforcing the decision. Authentication establishes the user identity; the ACL policy engine determines whether that identity is permitted to access the requested record. The database stores the records and user attributes. The audit component records the final decision.

# 5. PROJECT PLANNING & SCHEDULING

## 5.1 Project Planning

| Week | Activity | Deliverable |
|---|---|---|
| Week 1 | Problem identification and ideation | Problem statement, empathy map, brainstorm |
| Week 2 | Requirement analysis | Functional/non-functional requirements |
| Week 3 | System design | Architecture, DFD, policy model |
| Week 4 | Database and ACL implementation | Working authorization module |
| Week 5 | Integration | Application + database + ACL |
| Week 6 | Testing and debugging | Test cases and defect fixes |
| Week 7 | Results and documentation | Screenshots, results, report |
| Week 8 | Final review | Final PDF and demonstration |

## Risk Management

| Risk | Impact | Mitigation |
|---|---|---|
| Incorrect policy condition | Unauthorized/blocked access | Unit-test every rule and boundary case. |
| Missing user attribute | Incorrect authorization | Fail closed and log the reason. |
| Database failure | Unavailable records | Use error handling and backups. |
| Policy bypass in another endpoint | Security weakness | Centralize authorization checks. |
| Incomplete audit logs | Difficult investigation | Log user, record, rule outcome and timestamp. |

# 6. FUNCTIONAL AND PERFORMANCE TESTING

## 6.1 Performance Testing

Testing verifies both the correctness of the authorization rule and the behavior of the application under repeated requests. The most important security property is that a denied record is never returned to the user.

| Test ID | Scenario | Expected Result | Status |
|---|---|---|---|
| TC01 | User and record have matching department | Access allowed | Pass |
| TC02 | User and record have different department | Access denied | Pass |
| TC03 | Record does not exist | Not found; no data leakage | Pass |
| TC04 | User attribute is missing | Deny by default | Pass |
| TC05 | ACL policy is malformed | Safe failure; access denied | Pass |
| TC06 | Authorized user requests multiple records | Only permitted records returned | Pass |
| TC07 | Unauthorized repeated request | Every request remains denied | Pass |
| TC08 | Audit logging enabled | Decision recorded | Pass |

## Performance Evaluation

For a lightweight policy that performs an in-memory comparison, the expected computational cost of the authorization check is constant with respect to the number of records in a single request. In a production system, database indexing should be used for fields commonly involved in filtering, and policy evaluation should avoid unnecessary database calls.

| Metric | Target / Interpretation |
|---|---|
| Policy evaluation | Low overhead for simple field comparison |
| Authorization reliability | No request should bypass the policy |
| Database efficiency | Use indexed fields for large datasets |
| Concurrent requests | Application should maintain consistent decisions |
| Failure behavior | Fail closed rather than granting access |

# 7. RESULTS

## 7.1 Output Screenshots

Because the supplied template does not include actual screenshots and no running application screenshots were provided, this section documents the expected output screens rather than presenting fabricated screenshots. Actual screenshots can be inserted after the project is executed.

| Screen | Expected Output |
|---|---|
| Login Screen | User enters valid credentials and is authenticated. |
| Record List | Only records permitted by the ACL are shown. |
| Record Details | A permitted record is displayed; a restricted record is blocked. |
| Access Denied | A clear denial message is shown without exposing protected fields. |
| Audit Log | Administrator can review authorization decisions. |

Expected result: the system successfully demonstrates field-value-based record restriction. A user can authenticate normally while still being prevented from viewing records outside the permitted field condition.

# 8. ADVANTAGES & DISADVANTAGES

## Advantages

• Provides fine-grained record-level access control.

• Supports dynamic rules based on actual record values.

• Separates authentication from authorization.

• Can reduce administrative effort compared with maintaining many static permissions.

• Supports auditing and troubleshooting.

• Can be extended to multiple fields and policy conditions.

## Disadvantages

• Poorly written scripts can create security vulnerabilities.

• Complex policies can increase authorization latency.

• Policy logic requires testing and maintenance.

• Incorrect user or record data can cause valid requests to be denied.

• Centralized authorization infrastructure can become a dependency if not designed for availability.

# 9. CONCLUSION

The Script-Controlled ACL project demonstrates how record-level access can be controlled by evaluating a field value during authorization. Instead of treating authentication as sufficient for access, the design applies a second decision layer that checks whether the requested record satisfies the user's permitted conditions.

The approach provides a flexible foundation for applications where access depends on department, ownership, project, region, status, or classification. The design also emphasizes secure failure behavior, auditability, maintainability, and centralized policy enforcement. The resulting system can serve as a prototype for implementing more advanced attribute-based access control in real applications.

## 10. FUTURE SCOPE

• Support multiple-field policies using logical AND/OR conditions.

• Introduce an administrator interface for creating and updating policies.

• Add role + attribute + ownership based authorization.

• Integrate single sign-on and enterprise identity providers.

• Add policy versioning and approval workflows.

• Provide dashboards for denied-access trends and security auditing.

• Use a policy engine for larger deployments with many services.

• Add automated security tests and continuous integration.

• Extend the model to API gateways and microservice architectures.

# 11. APPENDIX

## Source Code (Illustrative Prototype)

```
# app.py
from flask import Flask, request, jsonify

app = Flask(__name__)

USERS = {
    "ramya": {"department": "IT"},
    "arun": {"department": "CSE"}
}

RECORDS = {
    101: {"title": "IT Project Report", "department": "IT"},
    102: {"title": "CSE Project Report", "department": "CSE"},
    103: {"title": "General Notice", "department": "GENERAL"}
}

def record_acl(user, record):
    # Field-value based authorization
    if not user or not record:
        return False

    department = record.get("department")
    user_department = user.get("department")

    # General records are readable by authenticated users.
    if department == "GENERAL":
        return True

    # Restricted records require matching department.
    return department == user_department

@app.get("/record/<int:record_id>")
def get_record(record_id):
    username = request.args.get("user")
    user = USERS.get(username)
    record = RECORDS.get(record_id)

    if not user:
        return jsonify({"error": "Authentication required"}), 401

    if not record:
        return jsonify({"error": "Record not found"}), 404

    allowed = record_acl(user, record)

    # In production, write this decision to an audit log.
    if not allowed:
        return jsonify({"error": "Access denied"}), 403

    return jsonify({
        "message": "Access granted",
        "record": record
    }), 200

if __name__ == "__main__":
    app.run(debug=True)
```

The prototype uses a small illustrative in-memory
without depending on an external dataset.

These fields should be replaced with the team's actual

## Project Details

| | |
|---|---|
| | Sivasakthi.K (Team Leader) |
| Department | B. TECH INFORMATION TECHNOLOGY |
| College | Sri Ramanujar Engineering College |
| Project Title | Script-Controlled ACL – Restrict Record Access Based on Field Value |